

ব্যক্তিগত পাঠ্যবই (উদাহরণ) · তথ্যবিজ্ঞান

Git, ভার্সনিং এবং কৃত্রিম বুদ্ধিমত্তা

শিক্ষার্থীর উদাহরণ

নাম গোপন উদাহরণ · বইটি শিক্ষার্থীর মাতৃভাষায়

বাংলা সংস্করণ — Git, ভার্সনিং এবং কৃত্রিম বুদ্ধিমত্তা

১. তিনটি টুল এবং তাদের সম্পর্ক

- Git এবং GitHub:** কাজ সুসংহতভাবে সংরক্ষণের ভাণ্ডার; প্রতিটি সংস্করণ সংরক্ষিত থাকে এবং শেয়ার করা যায়।
- Gemini:** একটি জেনারেটিভ কৃত্রিম বুদ্ধিমত্তা (AI) যার সাথে কথা বলা যায়; তুমি একটি লেখা (এমনকি একটি .md ফাইল) দাও, সে ব্যাখ্যা করে, অনুবাদ করে, সারসংক্ষেপ করে।
- NotebookLM:** একজন শিক্ষক যে কেবল তোমার আপলোড করা নথির (যেমন একটি PDF) ভিত্তিতে উত্তর দেয়, তাই সে উৎসের প্রতি বিশ্বস্ত থাকে।

মূল কথা: **Git সংরক্ষণ করে, Gemini ব্যাখ্যা করে, NotebookLM উৎসের ওপর পুনরালোচনা করায়।** এগুলো প্রতিযোগী নয়, একসাথে ব্যবহার করা হয়।

সোনালি নিয়ম: AI তোমাকে আরও ভালোভাবে **বোঝায়** সাহায্য করে, নকল করতে বা চিন্তা এড়াতে নয়।

পরীক্ষা: যদি নিজের ভাষায় বলতে পারো তুমি কী করেছ ও কেন, তবেই সত্যিই শিখেছ।

২. Git এবং GitHub

- Git** একটি **ভার্সন নিয়ন্ত্রণ** ব্যবস্থা: কাজের ইতিহাস রাখে, তাই প্রতিটি গুরুত্বপূর্ণ সংরক্ষণ থেকে যায় এবং পেছনে ফেরা যায়।
- GitHub** হলো সেই ওয়েবসাইট যেখানে ভার্সনকৃত কাজ থাকে ও শেয়ার হয়: অর্থাৎ "ইন্টারনেটে Git"। মনে রেখো: Git হলো **টুল (ইঞ্জিন)**, GitHub হলো **অনলাইন সেবা**।
- রিপোজিটরি:** প্রকল্পের ফোল্ডার, যাতে সব ফাইল ও তাদের পূর্ণ ইতিহাস থাকে।
- কমিট (commit):** লেবেলসহ সংরক্ষণ — একটি মুহুর্তে প্রকল্পের "ছবি" ও একটি বার্তা। পরপর অনেক কমিট মিলে প্রকল্পের ইতিহাস।
- ব্রাঞ্চ (branch):** একটি সমান্তরাল কাজের লাইন, মূল লাইন (main) নষ্ট না করে চেষ্টা করার জন্য; মূলে যুক্ত করাকে **merge** বলে।
- পুল রিকোয়েস্ট (Pull Request):** নিজের ব্রাঞ্চকে মূল লাইনে যুক্ত করার প্রস্তাব, সাধারণত কেউ আগে পর্যালোচনা করে।

সেবার ধরন **PaaS** (প্ল্যাটফর্ম অ্যাজ আ সার্ভিস): কেউ তোমাকে প্রস্তুত অনলাইন প্ল্যাটফর্ম দেয়, তুমি সার্ভার বানানো ছাড়াই ব্যবহার করো। মূল্য মডেল **Freemium**: মৌলিক ফিচার বিনামূল্যে, উন্নত ফিচার অর্থের বিনিময়ে; শিক্ষার্থীর

জন্য বিনামূল্যের অংশই যথেষ্ট।

৩. ভার্সিনিং: সংস্করণ নম্বর 1.2.3

1. **major (প্রথম সংখ্যা):** বড়, সম্ভবত পুরনোর সাথে অসামঞ্জস্যপূর্ণ পরিবর্তনে বাড়ে।
2. **minor (দ্বিতীয় সংখ্যা):** নতুন ফিচার যোগ হলে কিন্তু বাকিটা কাজ করলে বাড়ে।
3. **patch (তৃতীয় সংখ্যা):** শুধু একটি ত্রুটি সংশোধনে, নতুন কিছু ছাড়া বাড়ে।

উদাহরণ: v1.0.0 → শব্দ যোগ v1.1.0 → ত্রুটি সংশোধন v1.1.1 → সব নতুন করে v2.0.0।

সোনালি নিয়ম: প্রকাশিত সংস্করণ কখনো পুনরায় ব্যবহার হয় না। একটি জিনিস বদলালেও আগে নম্বর বাড়াও, তারপর প্রকাশ করো।

৪. Git-এর ব্যবহারের ক্ষেত্র (শব্দে বর্ণিত)

1. **রৈখিক উন্নয়ন:** একজন, ব্রাঞ্চ ছাড়া, কমিট একের পর এক, ইতিহাস সরলরেখা।
2. **এক ফিচার এক ব্রাঞ্চে, তারপর merge:** মূল লাইন সুস্থ থাকে; প্রস্তুত হলে যুক্ত করো।
3. **দুই ফিচার সমান্তরালে:** ভিন্ন সময়ে merge; মূল লাইন ধাপে ধাপে সংগ্রহ করে।
4. **জরুরি সংশোধন (hotfix):** বড় কাজ চলাকালীন গুরুতর ত্রুটি — ছোট ব্রাঞ্চে দ্রুত সংশোধন করে যুক্ত করো।
5. **প্রকাশিত সংস্করণ:** মূল লাইনে সংস্করণ প্রকাশ (v1.0, v1.1... বড় পরিবর্তনে v2.0); প্রতিটি প্রকাশ "হিমায়িত"।
6. **পরিত্যক্ত ব্রাঞ্চ:** একটি ধারণা পরীক্ষা করে যুক্ত না করা; মূল লাইন অক্ষত থাকে।
7. **এক ব্রাঞ্চ থেকে আরেক ব্রাঞ্চ:** বড় ফিচার সাব-ব্রাঞ্চ বের করে; ভেতর থেকে বাইরে যুক্ত হয়।
8. **বড় সমন্বয় (দলগত প্রকল্প):** প্রত্যেকের এক ব্রাঞ্চ, শেষে Pull Request দিয়ে মূলে যুক্ত হয়ে পূর্ণ সংস্করণ।
9. **দীর্ঘমেয়াদি রক্ষণাবেক্ষণ ব্রাঞ্চ:** মূল লাইন v2.0-এর দিকে, তবু v1 ব্যবহারকারীদের জন্য শুধু সংশোধনের সমান্তরাল ব্রাঞ্চ।
10. **revert:** ইতিহাস মোছা হয় না; একটি নতুন কমিট ভুলটি বাতিল করে, সততার সাথে চিহ্ন রেখে।
11. **merge দ্বন্দ্ব সমাধান:** দুই ব্রাঞ্চ একই লাইন বদলালে Git থেমে যায়, একজন সিদ্ধান্ত নেয় কোনটি রাখা হবে।
12. **পাঁচ ব্রাঞ্চ ও দুই প্রকাশের দল:** মূল লাইন প্রকাশ করে, চারটি ব্রাঞ্চ সমান্তরালে চলে।
13. **পাঁচ স্তরের নেস্টেড ব্রাঞ্চ:** জটিল কাজ স্তরে স্তরে বিভক্ত, প্রতিটি স্তর পূর্ববর্তীতে ফেরে।
14. **দুই প্রধান সংস্করণ একসাথে:** v1.x ও v2.x-এর জন্য দুটি রক্ষণাবেক্ষণ ব্রাঞ্চ একসাথে।
15. **বাস্তব ইতিহাস:** প্রকাশ, ফিচার+সাব-ব্রাঞ্চ, hotfix, revert, দ্বন্দ্বসহ merge — সব একসাথে।

৫. Markdown ফাইল (.md)

1. Markdown (.md) হলো **বিশুদ্ধ টেক্সট**, অল্প চিহ্ন দিয়ে বিন্যাস: # শিরোনাম, ** গাঢ়, 1. দিয়ে সংখ্যাযুক্ত তালিকা।
2. সর্বত্র পড়া যায়, ভারী ছবি নেই, শুধু বিষয়বস্তু। তাই AI এটি ভালোভাবে পড়ে: আমরা নোটগুলো .md আকারেও দিই যাতে Gemini-কে দেওয়া যায়।

৬. Gemini: জেনারেটিভ AI, এবং .md ফাইল দিয়ে ব্যবহার

1. Gemini একটি **জেনারেটিভ** AI: তুমি একটি অনুরোধ (prompt) লেখো, সে উত্তর তৈরি করে — ব্যাখ্যা, অনুবাদ, সারসংক্ষেপ, পুনরালোচনার প্রশ্ন।
2. সতর্কতা: AI ভুল করতে বা বানিয়ে বলতে পারে ("হ্যালুসিনেশন"); বিশেষত তারিখ, সংখ্যা ও উৎস যাচাই করো।
3. ধাপ: ব্রাউজারে gemini.google.com (স্কুল অ্যাকাউন্ট); .md ফাইল খুলে সব নির্বাচন করে কপি; Gemini-তে পেস্ট করে অনুরোধ লেখো।

কোন সংস্করণ: **Flash** (দ্রুত, দৈনন্দিন), **Pro** (শক্তিশালী, জটিল লেখা), **রিজনিং** (উত্তরের আগে "ভাবে", কঠিন সমস্যায়)। নিয়ম: Flash দিয়ে শুরু করো; জটিল বা ভুল মনে হলে Pro বা রিজনিং।

৭. NotebookLM: উৎসভিত্তিক শিক্ষক, এবং PDF দিয়ে ব্যবহার

1. NotebookLM কেবল আপলোড করা নথির ভিত্তিতে উত্তর দেয়: একটি PDF দিলে সেই PDF থেকেই উত্তর, বাইরে বানায় না।
2. তাই নির্দিষ্ট উৎসে পড়া ও পুনরালোচনার জন্য চমৎকার, উৎসের প্রতি বিশ্বস্ত।
3. ধাপ: notebooklm.google.com (স্কুল অ্যাকাউন্ট); "নতুন তৈরি" → "আপলোড" → একটি PDF; চ্যাটে প্রশ্ন করো, যেমন "এই নথির একটি সহজ সারসংক্ষেপ দাও"।

৮. সম্পূর্ণ প্রবাহ: তিনটি পরিবেশ একসাথে

1. কোর্সের উপকরণ **Git/GitHub**-এ (ভার্সনকৃত)।
2. সেখান থেকে PDF (পড়া/মুদ্রণ) বা .md (AI-এর জন্য) নামাও।
3. .md দাও **Gemini**-কে ব্যাখ্যা/অনুবাদের জন্য।
4. PDF আপলোড করো **NotebookLM**-এ প্রশ্ন ও পুনরালোচনার জন্য।
5. সম্পন্ন কাজ **Classroom**-এ জমা দাও।

৯. AI-এর দায়িত্বশীল ব্যবহার

1. **বোঝা, নকল নয়:** AI শেখার সহায়, কাজ এড়ানোর পথ নয়। পরীক্ষা: নিজের ভাষায় বলতে পারা।
2. **সর্বদা যাচাই:** AI ভুল করতে পারে; যা বলে তা যাচাই করো।
3. **গোপনীয়তা ও তথ্য:** ব্যক্তিগত, সহপাঠী বা পরিবারের তথ্য এসব সহায়ককে দিও না।

মূল্যায়নের উদাহরণ: নম্বর ৯০/১০০ — কাজ সম্পূর্ণ ও সঠিক, Git ও ভার্সিনিং বিষয়ে স্পষ্ট বোঝাপড়া এবং নিজের ভাষায় ব্যাখ্যার সামর্থ্য। (নাম গোপন উদাহরণ)